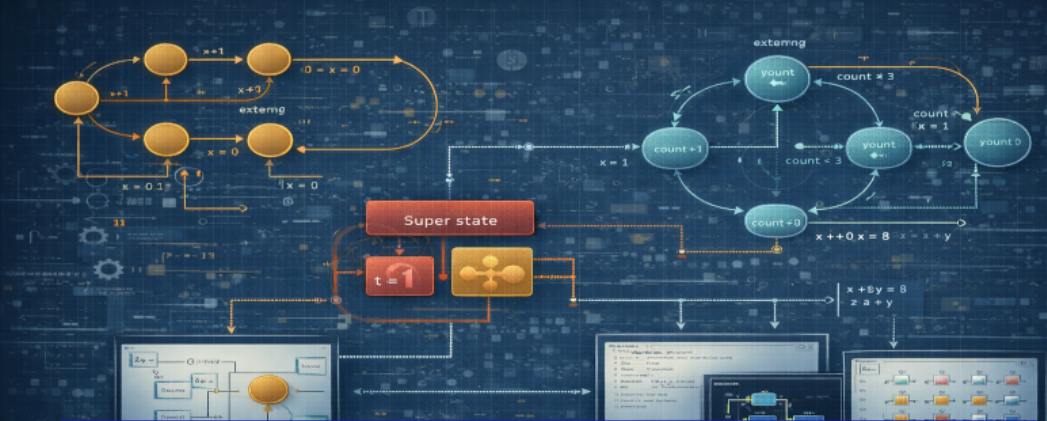




8. Modelagem de Sistemas Lógicos por Máquinas de Estados Finitos

8.1 Introdução

No capítulo anterior foram tratados modelos contínuos de sistemas físicos por equações diferenciais. Entretanto, muitos sistemas relevantes em engenharia e computação não são descritos apenas por variáveis contínuas. Controladores, protocolos, sequenciadores, supervisores, módulos de interface, rotinas embarcadas e lógicas de decisão operam, em geral, por mudanças discretas de modo, disparadas por eventos, comparações, temporizações ou comandos externos. Nesses casos, o formalismo mais natural não é o das equações diferenciais, mas o das **máquinas de estados finitos**.

Esse tipo de modelagem é particularmente importante em automação, sistemas embarcados, eletrônica digital, robótica, protocolos de comunicação, supervisão de processos, bioengenharia e sistemas ciberfísicos em geral. Em todos esses domínios, a parte lógica do sistema responde a sinais, condições e eventos, decidindo *quando* mudar de modo de operação e *como* reagir ao ambiente ou a outros subsistemas. Assim, enquanto os modelos contínuos descrevem a dinâmica física, os modelos por estados discretos descrevem a dinâmica lógica.

Contudo, uma máquina de estados finitos clássica nem sempre é suficiente. Em problemas reais, a lógica de controle frequentemente depende de variáveis auxiliares, contadores,

temporizações, prioridades, refinamentos internos e comunicação entre múltiplas máquinas. Por essa razão, o desenvolvimento deste capítulo partirá das máquinas de estados clássicas, mas avançará até **máquinas de estados finitos estendidas**, máquinas **hierárquicas** e máquinas **comunicantes**. Essa progressão é importante porque ela reflete o modo como controladores reais são projetados e implementados.

Também é importante observar que este capítulo deve preparar diretamente o capítulo seguinte. A modelagem híbrida de sistemas ciberfísicos exige que a lógica discreta e a dinâmica contínua possam coexistir de forma coerente. Portanto, aqui a ênfase recai em controladores e comportamentos discretos descritos por estados e transições, enquanto no próximo capítulo esses elementos serão acoplados a plantas contínuas. Por essa razão, os exemplos principais do capítulo serão organizados em torno de dois casos já compatíveis com essa futura integração: o controle ON-OFF da temperatura de um ambiente e o controle de um semáforo temporizado.

Do ponto de vista computacional, este capítulo utilizará dois ambientes de modelagem e simulação: *Ptolemy II* e *OpenModelica*. Essa escolha não é casual. O *Ptolemy II* oferece suporte particularmente claro à representação explícita de máquinas de estados e modelos modais, enquanto o *OpenModelica* permite descrever *state machines* no mesmo ambiente em que serão modelados subsistemas físicos contínuos. Essa dupla abordagem é metodologicamente valiosa, pois mostra duas formas diferentes, mas compatíveis, de materializar o mesmo formalismo.

8.2 Definições e Conceituação

8.2.1 Máquinas de estados finitos clássicas

Uma **máquina de estados finitos** pode ser entendida como um sistema discreto cuja evolução é descrita por um conjunto finito de estados e por transições entre esses estados. Em cada instante lógico, a máquina ocupa exatamente um estado, reage às entradas disponíveis e, conforme sua semântica, pode permanecer no estado atual ou transitar para outro.

Uma forma abstrata de representar uma FSM consiste na tupla

$$\Gamma = (\mathbb{S}, \mathbb{E}, \mathbb{T}, s_0, G, h, F),$$

em que:

- $\mathbb{S}$ é o conjunto finito de estados;
- $\mathbb{E}$ é o alfabeto de entradas;
- $\mathbb{T}$ é o alfabeto de saídas;
- $s_0 \in \mathbb{S}$ é o estado inicial;
- G é a função de próximo estado;
- h é a função de saída;
- $F \subseteq \mathbb{S}$, quando fizer sentido, um conjunto de estados finais.

Se s_t representa o estado atual e v_t representa a entrada disponível na reação atual, então

a função de próximo estado pode ser escrita como

$$s_{t+1} = G(s_t, v_t).$$

De modo análogo, a função de saída pode ser escrita, em formulação geral, como

$$\eta_t = h(s_t, v_t).$$

O elemento conceitual mais importante aqui é que a memória relevante da máquina está resumida no estado atual. Isso significa que, para efeitos de evolução futura, não é necessário armazenar toda a história passada do comportamento da máquina; basta conhecer seu estado corrente e as entradas presentes. Essa propriedade explica por que máquinas de estados são tão importantes na modelagem de controladores, protocolos e sistemas reativos.

A representação mais comum de uma FSM é o diagrama de transição de estados. Na figura 8.1 pode-se observar um exemplo genérico desse tipo de diagrama. Os nós representam estados, enquanto as arestas orientadas representam as transições possíveis.

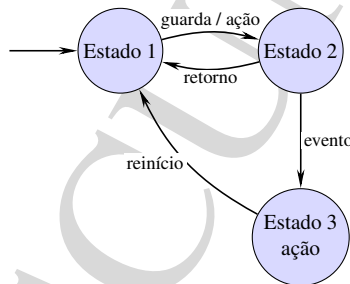


Figura 8.1: Diagrama genérico de transição de estados de uma máquina de estados finitos, destacando estado inicial, estados intermediários e transições rotuladas.

8.2.2 Máquinas de Moore e de Mealy

Do ponto de vista da função de saída, é clássica a distinção entre máquinas de Moore e máquinas de Mealy. Em uma **máquina de Moore**, a saída depende apenas do estado atual:

$$\eta_t = h(s_t).$$

Nesse caso, o valor de saída está associado ao próprio estado, e isso é frequentemente representado graficamente dentro do nó correspondente.

Em uma **máquina de Mealy**, a saída depende do estado atual e da entrada presente:

$$\eta_t = h(s_t, v_t).$$

Nesse caso, a saída é mais naturalmente associada à transição, pois depende da situação em que a mudança de estado ocorre. Graficamente, isso costuma aparecer em rótulos do tipo

guarda/ação.

Essa distinção não é apenas notacional. Máquinas de Moore tendem a ser mais fáceis de interpretar, enquanto máquinas de Mealy frequentemente produzem modelos mais compactos e respostas logicamente mais imediatas às entradas. Ao longo deste capítulo, utilizaremos uma notação suficientemente geral para acomodar ambos os estilos.

8.2.3 Transições, guardas e ações

Uma transição de estado pode conter, conceitualmente, três elementos:

1. uma **guarda**, que determina quando a transição está habilitada;
2. uma **ação de saída**, que define valores produzidos durante a transição;
3. e, em máquinas estendidas, uma **ação de atualização**, que altera variáveis internas.

A guarda é uma expressão Booleana. Se ela for verdadeira e a semântica da máquina permitir, a transição pode ser disparada. Se não houver guarda explicitada, assume-se, em muitos formalismos, que a transição está sempre habilitada. Já as ações associadas à transição podem representar comandos emitidos, sinais de saída ou atualizações de memória auxiliar.

Essa separação entre habilitação e efeito é importante. Ela distingue claramente:

- a condição sob a qual a máquina pode mudar de estado;
- o efeito observável produzido nessa mudança;
- e, quando necessário, a alteração de variáveis internas que participarão de decisões futuras.

8.2.4 Determinismo, receptividade e completude

Uma FSM é dita **determinística** quando, para um dado estado atual e uma dada situação de entrada, existe no máximo uma transição habilitada. Nessa condição, a evolução da máquina fica univocamente determinada. Se mais de uma transição puder ser habilitada simultaneamente na mesma situação, a máquina apresenta **não determinismo**.

Uma máquina é dita **receptiva** quando, para qualquer situação admissível de entrada, existe algum comportamento possível da máquina, isto é, pelo menos uma transição executável ou uma forma coerente de permanência. Já a **completude** diz respeito ao fato de o modelo cobrir explicitamente todos os casos relevantes de entrada em todos os estados.

Essas propriedades têm implicações práticas diretas. Em uma implementação executável, não determinismo mal controlado pode tornar o comportamento difícil de prever. Falta de receptividade pode produzir travamentos lógicos. Incompletude pode indicar que o projeto deixou casos relevantes sem tratamento.

8.2.5 Máquinas de estados finitos estendidas

Máquinas de estados finitos clássicas são adequadas quando o comportamento do sistema pode ser descrito apenas pelo estado atual e pelas entradas presentes. Entretanto, muitos sistemas reais exigem memória auxiliar explícita. É nesse ponto que surgem as **máquinas de estados finitos estendidas** (FSMe).

Em uma FSMe, além do estado de controle, o comportamento depende também de um conjunto de variáveis internas

$$z = (z_1, z_2, \dots, z_m).$$

Com isso, a evolução pode ser descrita por expressões do tipo

$$s_{t+1} = G(s_t, v_t, z_t), \quad z_{t+1} = U(s_t, v_t, z_t), \quad \eta_t = h(s_t, v_t, z_t).$$

A presença de z_t amplia o espaço efetivo de estados do sistema, mas sem exigir que todas as combinações possíveis sejam desenhadas explicitamente no grafo. Isso é metodologicamente valioso porque evita explosão combinatória. Em vez de multiplicar estados para representar contadores, temporizações, modos internos e parâmetros, essas informações podem ser mantidas como memória auxiliar.

Esse é o caso, por exemplo, de controladores temporizados, semáforos, protocolos com contadores de tentativas, supervisores com modo de falha e sistemas com histerese. Em todos esses casos, uma FSM clássica pura pode se tornar artificialmente grande ou pouco legível, enquanto uma FSMe oferece abstração mais natural.

8.2.6 Hierarquia e composição concorrente

Outra limitação frequente de diagramas planos é a dificuldade de organizar controladores grandes. Para isso, introduz-se **hierarquia**. Em uma máquina hierárquica, certos estados deixam de ser atômicos e passam a encapsular internamente outro submodelo, frequentemente outra máquina de estados ou um comportamento refinado. Isso permite separar níveis de abstração: no nível superior, aparecem apenas os modos globais do sistema; no nível inferior, os detalhes internos de cada modo.

Além disso, muitos sistemas lógicos não são puramente sequenciais. Em aplicações reais, controladores locais, supervisores, módulos de interface e protocolos podem operar em paralelo, trocando sinais e coordenando comportamento. Para representar esse tipo de situação, utiliza-se a **composição concorrente** de máquinas comunicantes. A figura 8.2 ilustra essa ideia.

Essa composição pode ser síncrona ou assíncrona, dependendo da semântica adotada pelo ambiente de modelagem. No contexto deste capítulo, o foco principal será a composição síncrona, especialmente útil para integração posterior com modelos híbridos.

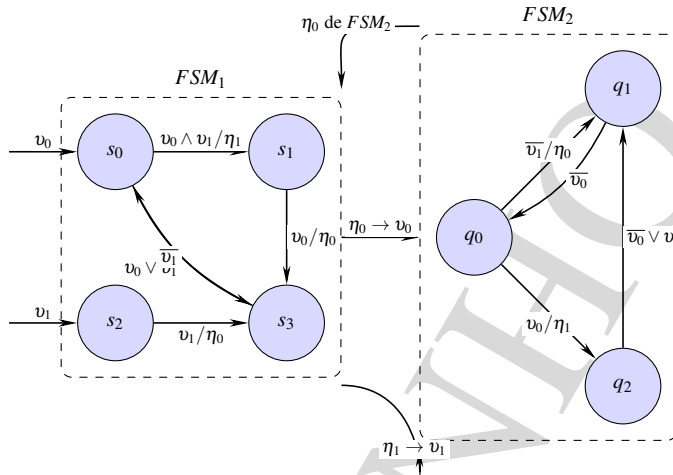


Figura 8.2: Exemplo de composição concorrente de máquinas de estados comunicantes, em que cada submáquina possui estado local próprio e o comportamento global emerge da troca disciplinada de sinais.

Exemplo 8.1 Explique por que a introdução de uma variável contador em uma máquina de estados pode ser preferível à criação de vários estados adicionais explícitos.

Solução: Porque o contador permite representar memória auxiliar sem multiplicar artificialmente o número de estados de controle. Se cada valor do contador fosse codificado como estado distinto, o diagrama cresceria rapidamente e perderia clareza. Em uma FSM, o estado lógico principal permanece compacto, enquanto a informação quantitativa fica representada por variável interna. ■

8.3 Modelagem de Sistemas Lógicos

8.3.1 Sinais discretos e semântica reativa

Em muitos sistemas lógicos, as entradas e saídas são mais bem entendidas como **sinais discretos**. Esses sinais podem representar presença de evento, comandos binários, símbolos, condições lógicas, mensagens ou amostras de grandezas contínuas já discretizadas no tempo. Em termos gerais, pode-se pensar em sequências

$$v = (v_0, v_1, v_2, \dots), \quad \eta = (\eta_0, \eta_1, \eta_2, \dots),$$

nas quais cada reação lógica produz uma nova resposta da máquina.

Isso é particularmente importante em sistemas ciberfísicos. Embora a planta física evolua continuamente, o controlador frequentemente opera sobre amostras, comparações e comandos discretos. Assim, a fronteira entre o contínuo e o discreto não é ontológica, mas funcional: um mesmo sistema real pode conter subsistemas contínuos e subsistemas lógicos reagindo em instantes discretos.

8.3.2 Exemplo 1: controlador ON-OFF da temperatura

Retomemos o sistema térmico discutido no capítulo anterior. Ali, a planta física era descrita por uma equação diferencial contínua, enquanto o aquecedor era comandado por uma variável binária $\mu \in \{0, 1\}$. O problema lógico agora consiste em decidir o valor desse comando com base na temperatura atual T e no valor de referência T_{sp} .

Um modelo elementar pode ser descrito por uma máquina com dois estados:

- **Off**, em que o aquecedor permanece desligado;
- **On**, em que o aquecedor permanece ligado.

A lógica básica é:

- se $T \leq T_{sp}$, a máquina deve transitar ou permanecer em **On**;
- se $T \geq T_{sp}$, a máquina deve transitar ou permanecer em **Off**.

A figura 8.3 apresenta esse controlador. O aspecto principal a observar é que a lógica do aquecedor pode ser descrita inteiramente por uma FSM clássica simples.

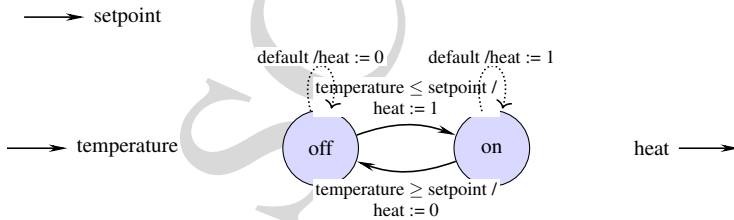


Figura 8.3: Máquina de estados do controlador ON-OFF da temperatura do ambiente, com estados **Off** e **On** e transições condicionadas pela comparação entre temperatura e *set-point*.

Esse exemplo é metodologicamente importante por três razões. Primeiro, ele mostra que uma lógica de controle simples pode ser descrita por apenas dois estados. Segundo, ele evidencia como uma variável contínua da planta aparece como entrada para a máquina discreta. Terceiro, ele prepara diretamente a integração futura entre planta e controlador no capítulo de sistemas híbridos.

Contudo, essa forma simples possui uma limitação prática conhecida: ela pode produzir comutação excessiva quando a temperatura oscila em torno do valor de referência. Para resol-

ver isso, pode-se introduzir histerese. Nesse caso, a lógica passa a depender não apenas da temperatura, mas também do estado anterior do aquecedor, o que nos conduz naturalmente a uma máquina estendida. Com dois limiares T_{low} e T_{high} , pode-se escrever

$$heat_{k+1} = \begin{cases} 1, & T_k \leq T_{low}, \\ 0, & T_k \geq T_{high}, \\ heat_k, & T_{low} < T_k < T_{high}. \end{cases}$$

Nesse caso, a memória do sistema não está apenas no estado de controle desenhado, mas também na permanência do comando anterior.

8.3.3 Exemplo 2: semáforo temporizado

Como segundo exemplo, considere o controle lógico de um semáforo. Os modos principais de operação podem ser descritos por três estados:

- Green;
- Yellow;
- Red.

Entretanto, a permanência em cada estado depende de duração temporal. Se tentássemos codificar cada unidade de tempo como estado separado, o diagrama cresceria desnecessariamente. Por isso, esse problema é mais bem representado por uma FSMe com variável interna *timer*, atualizada a cada reação lógica.

Transições típicas podem ser descritas por guardas como

$$timer \geq T_G, \quad timer \geq T_Y, \quad timer \geq T_R,$$

e por atribuições do tipo

$$timer := 0$$

sempre que uma nova fase é iniciada. A figura 8.4 ilustra essa ideia.

Esse exemplo mostra claramente por que a extensão do formalismo é importante. Os estados continuam representando modos qualitativos do sistema, mas a lógica de mudança depende de memória quantitativa auxiliar.

Exemplo 8.2 Considere um semáforo com estados Green, Yellow e Red, e uma variável interna *timer*. Suponha que a troca de fase ocorra quando *timer* atinge, respectivamente, 5, 2 e 4 unidades lógicas em cada estado. Explique por que esse sistema é mais naturalmente representado por uma FSMe do que por uma FSM clássica pura.

Solução: Porque a decisão de mudança de estado depende explicitamente de uma variável interna de contagem temporal. Se essa informação fosse representada apenas por estados, seria necessário criar múltiplos estados adicionais para cada valor de permanência em cada fase, o que tornaria o diagrama artificialmente grande. A FSMe permite manter os estados principais Green, Yellow e Red e deslocar a contagem temporal para uma

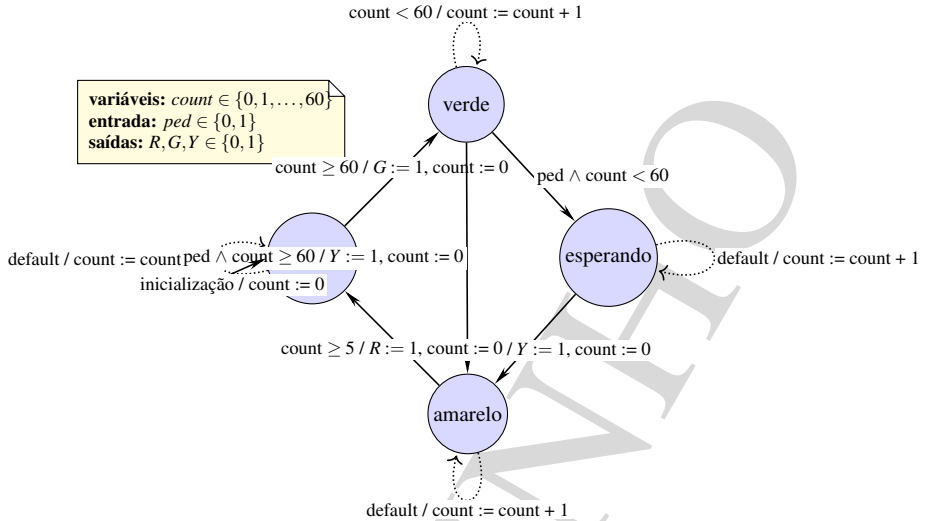


Figura 8.4: Máquina de estados finitos estendida para controle de semáforo, com estados principais de fase e transições condicionadas por variáveis internas de temporização.

variável auxiliar.

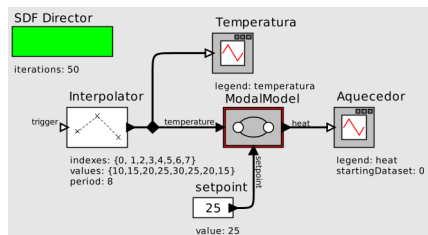
8.4 Simulação com Ptolemy II

O *Ptolemy II* é, entre as ferramentas utilizadas neste livro, aquela que oferece suporte mais direto e semanticamente explícito à modelagem por máquinas de estados. FSMs, máquinas estendidas e modelos modais podem ser construídos de forma natural por meio de *FSMACTOR* e *ModalModel*. Isso torna o ambiente particularmente adequado para representar, de forma visual e executável, as noções desenvolvidas nas seções conceituais.

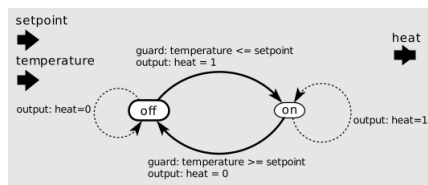
8.4.1 Controlador ON-OFF da temperatura em Ptolemy II

Uma forma didática de implementar o controlador ON-OFF é criar, no nível superior, um modelo com diretor discreto, um gerador artificial da temperatura, uma constante para o *set-point*, blocos de visualização e um *ModalModel* que representa o controlador lógico. A figura 8.5 mostra esse arranjo.

Do ponto de vista prático, a construção pode ser resumida nos seguintes passos:



(a) Modelo de nível superior



(b) FSM interna ao ModalModel

Figura 8.5: Modelo do controlador ON-OFF da temperatura em Ptolemy II. À esquerda aparece o modelo de nível superior, e à direita a FSM interna ao ModalModel.

1. criar um novo Graph Editor;
2. inserir um diretor adequado ao modelo discreto;
3. inserir um Interpolator ou outro gerador discreto para produzir a sequência de temperatura;
4. inserir uma constante setpoint;
5. inserir um ModalModel;
6. definir no ModalModel as portas de entrada temperature e setpoint, e a porta de saída heat;
7. abrir o ModalModel e construir internamente os estados Off e On, com as transições e ações correspondentes;
8. conectar o sinal de saída a um SequencePlot.

Ao executar a simulação, obtém-se o gráfico mostrado na figura 8.6. O ponto principal a observar é a correspondência entre a evolução da temperatura de entrada e a comutação do sinal heat.

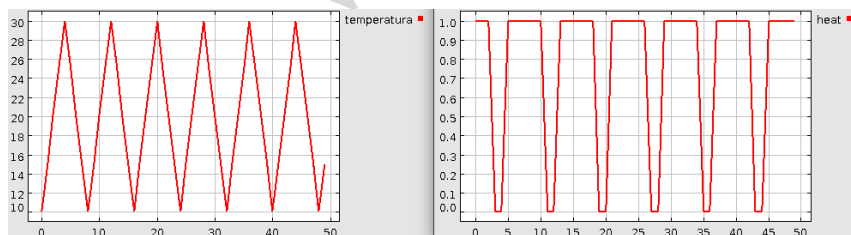


Figura 8.6: Resultado da simulação do controlador ON-OFF da temperatura em Ptolemy II, mostrando o sinal de entrada e o comando binário heat.

8.4.2 Semáforo temporizado em Ptolemy II

O segundo exemplo em Ptolemy II é o semáforo temporizado. Aqui, o interesse está em mostrar o uso de variáveis internas, guardas e ações de atualização, isto é, uma máquina de estados finitos estendida executável. A figura 8.7 ilustra uma implementação típica.

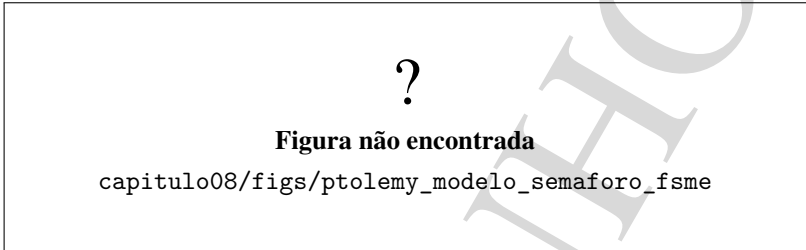


Figura 8.7: Implementação, em Ptolemy II, de uma máquina de estados finitos estendida para controle de semáforo.

Nesse tipo de modelo, guardas, *output actions* e *set actions* cumprem papéis distintos. As guardas determinam se a transição está habilitada; as *output actions* produzem sinais observáveis; e as *set actions* atualizam a memória interna da máquina. Essa separação é particularmente importante em Ptolemy II, pois torna explícita a distinção entre efeitos observáveis na reação corrente e atualização consolidada do estado interno.

8.4.3 Hierarquia e máquinas comunicantes

Uma vantagem adicional do Ptolemy II é a possibilidade de construir modelos modais hierárquicos e máquinas comunicantes. A figura 8.8 mostra uma FSM hierárquica, enquanto a figura 8.9 mostra uma composição concorrente de máquinas com troca de sinais.

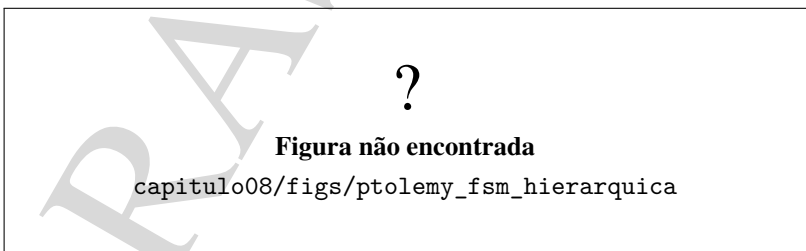


Figura 8.8: Exemplo de modelagem hierárquica em Ptolemy II, com estados que encapsulam refinamentos internos.

?

Figura não encontrada

capitulo08/figs/ptolemy_fsm_comunicantes

Figura 8.9: Exemplo de máquinas de estados comunicantes em Ptolemy II, organizadas em paralelo e trocando sinais de coordenação.

Esses recursos são especialmente importantes porque aproximam a modelagem lógica de problemas reais de automação e sistemas embarcados, nos quais o comportamento global raramente se reduz a uma única máquina plana.

8.5 Simulação com OpenModelica

O *OpenModelica* permite modelar máquinas de estados por meio dos recursos síncronos da linguagem Modelica. Diferentemente do Ptolemy II, em que a máquina aparece diretamente como artefato gráfico próprio, em *OpenModelica* a estrutura lógica é descrita por blocos, estados, transições e operadores declarativos da linguagem. Isso oferece uma segunda perspectiva metodológica bastante importante: a representação declarativa de sistemas lógicos no mesmo ambiente em que, nos capítulos 7 e 9, também serão descritos modelos contínuos e híbridos.

8.5.1 Elementos centrais das *state machines* em Modelica

Três elementos merecem destaque:

- `initialState(...)` define o estado inicial;
- `transition(...)` define transições entre estados;
- operadores como `timeInState()` e `ticksInState()` permitem expressar permanência temporal e reação lógica.

Essa estrutura torna o ambiente especialmente adequado para máquinas temporizadas, composições paralelas e integração com dinâmicas contínuas.

8.5.2 Controlador ON-OFF da temperatura em OpenModelica

Uma implementação textual possível do controlador ON-OFF da temperatura é apresentada na listagem 8.1. O objetivo aqui não é discutir todos os detalhes da linguagem síncrona de Modelica, mas mostrar como a mesma lógica de dois estados pode ser descrita declarativamente.

```
1  model ControleTemperaturaSM
2    input Real temperature;
3    parameter Real setpoint = 20;
4    output Boolean heat;
5
6    inner output Boolean heatSignal(start=false);
7
8    block OffState
9      outer output Boolean heatSignal;
10     equation
11       heatSignal = false;
12     end OffState;
13
14    block OnState
15      outer output Boolean heatSignal;
16     equation
17       heatSignal = true;
18     end OnState;
19
20    OffState offState;
21    OnState onState;
22
23    equation
24      heat = heatSignal;
25
26    initialState(offState);
27
28    transition(offState, onState, temperature <= setpoint,
29      immediate=true, reset=true, priority=1);
30
31    transition(onState, offState, temperature >= setpoint,
32      immediate=true, reset=true, priority=1);
33  end ControleTemperaturaSM;
```

Listing 8.1: Controlador ON-OFF da temperatura em OpenModelica

A figura 8.10 apresenta uma visualização correspondente no OMEdit.

A principal diferença em relação ao Ptolemy II não está na lógica do controlador, mas no estilo de modelagem. Aqui, a máquina é descrita declarativamente dentro da linguagem Modelica e pode coexistir no mesmo ambiente com componentes físicos contínuos.

8.5.3 Semáforo temporizado em OpenModelica

O semáforo temporizado é um exemplo particularmente natural para OpenModelica, pois o ambiente oferece recursos específicos para definir transições baseadas no tempo transcorrido no estado ativo. A listagem 8.2 ilustra uma implementação textual coerente com essa ideia.

?

Figura não encontrada

capitulo08/figs/openmodelica_controle_temperatura

Figura 8.10: Representação, em OpenModelica, do controlador ON-OFF da temperatura, com estados de aquecedor ligado e desligado.

```

1  model SemaforoSM
2  parameter Real TG = 5;
3  parameter Real TY = 2;
4  parameter Real TR = 4;
5
6  output Boolean green;
7  output Boolean yellow;
8  output Boolean red;
9
10 inner output Boolean gSig(start=true);
11 inner output Boolean ySig(start=false);
12 inner output Boolean rSig(start=false);
13
14 block GreenState
15   outer output Boolean gSig;
16   outer output Boolean ySig;
17   outer output Boolean rSig;
18   equation
19     gSig = true;
20     ySig = false;
21     rSig = false;
22   end GreenState;
23
24 block YellowState
25   outer output Boolean gSig;
26   outer output Boolean ySig;
27   outer output Boolean rSig;
28   equation
29     gSig = false;
30     ySig = true;
31     rSig = false;
32   end YellowState;
33
34 block RedState
35   outer output Boolean gSig;
```

```
36   outer output Boolean ySig;
37   outer output Boolean rSig;
38   equation
39     gSig = false;
40     ySig = false;
41     rSig = true;
42   end RedState;
43
44   GreenState greenState;
45   YellowState yellowState;
46   RedState redState;
47
48   equation
49     green = gSig;
50     yellow = ySig;
51     red = rSig;
52
53   initialState(greenState);
54
55   transition(greenState, yellowState, timeInState() >= TG,
56     immediate=false, reset=true, priority=1);
57   transition(yellowState, redState, timeInState() >= TY,
58     immediate=false, reset=true, priority=1);
59   transition(redState, greenState, timeInState() >= TR,
60     immediate=false, reset=true, priority=1);
61   end SemaforoSM;
```

Listing 8.2: Semáforo temporizado em OpenModelica

A figura 8.11 mostra a organização correspondente no ambiente gráfico.

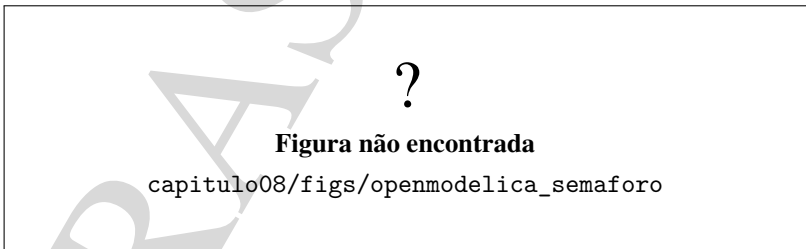


Figura 8.11: Modelo de semáforo temporizado em OpenModelica, com estados correspondentes às fases do sinal e transições condicionadas pelo tempo de permanência em cada estado.

8.5.4 Máquinas comunicantes em OpenModelica

O OpenModelica também permite compor máquinas em paralelo, com troca de sinais e sincronização disciplinada. Essa possibilidade é particularmente útil em arquiteturas de supervisão, controle distribuído e coordenação entre subsistemas lógicos. A figura 8.12 ilustra um caso representativo.

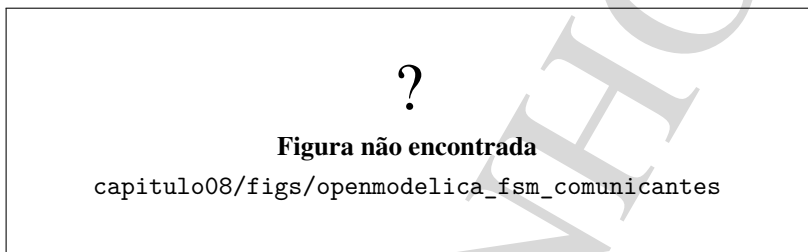


Figura 8.12: Exemplo de máquinas de estados comunicantes em OpenModelica, com uma máquina supervisora e uma máquina local trocando sinais de comando e estado.

Do ponto de vista metodológico, essa composição é importante porque mostra que a modelagem por estados não se limita a controladores isolados. Ela também permite descrever sistemas distribuídos de decisão, o que será particularmente relevante na integração com plantas físicas no capítulo seguinte.

8.6 Comparação metodológica entre Ptolemy II e OpenModelica

Os dois ambientes utilizados neste capítulo suportam modelagem por estados, mas enfatizam aspectos diferentes do processo. No *Ptolemy II*, máquinas de estados, modelos modais, refinamentos e composição concorrente são representados de forma explícita e visualmente muito direta. Isso o torna especialmente adequado para estudo conceitual e para exploração semântica de transições, guardas, hierarquia e comunicação.

No *OpenModelica*, por outro lado, a modelagem ocorre em uma estrutura declarativa mais próxima da linguagem do ambiente. Isso é especialmente vantajoso quando a máquina lógica precisa coexistir, no mesmo modelo e no mesmo ecossistema computacional, com subsistemas contínuos. Em outras palavras, enquanto o *Ptolemy II* oferece maior transparência visual para o formalismo lógico, o *OpenModelica* oferece integração particularmente forte com o universo da modelagem física declarativa.

Essa comparação é central para a lógica da Parte III. Os capítulos 7 e 8 mostram, separadamente, modelagem contínua e modelagem lógica. O capítulo 9 exigirá ambas em conjunto. Por isso, o uso dos mesmos dois ambientes aqui não é apenas escolha pedagógica: é uma

preparação metodológica deliberada para a modelagem híbrida.

8.7 Considerações Finais

As máquinas de estados finitos constituem um dos formalismos mais importantes para a representação da parte lógica de sistemas de engenharia e computação. Neste capítulo, mostrou-se que esse formalismo pode ser desenvolvido progressivamente, partindo das máquinas clássicas e avançando para máquinas estendidas, hierárquicas e comunicantes. Com isso, tornou-se claro que a escolha entre uma FSM simples e uma FSMe não é mera questão de estilo, mas depende da necessidade de representar memória auxiliar, temporização, comunicação e refinamento interno.

Também se evidenciou que o formalismo conceitual pode ser materializado de formas computacionais distintas sem perder seu significado lógico. O *Ptolemy II* mostrou-se particularmente forte na representação direta e visual da semântica das máquinas, enquanto o *OpenModelica* mostrou a força da descrição declarativa integrada a um ambiente também voltado à modelagem contínua. Essa dualidade é especialmente importante no contexto do livro, pois prepara a transição metodológica para sistemas híbridos.

Com isso, o próximo capítulo poderá integrar explicitamente os dois mundos já estudados: a dinâmica contínua de plantas físicas e a lógica discreta de controladores e supervisores. É precisamente dessa integração entre equações diferenciais e máquinas de estados que emergem os modelos híbridos de sistemas ciberfísicos.

8.8 Resumo

Seção 8.1 Introdução

- Máquinas de estados finitos são um formalismo central para descrever controladores, supervisores, protocolos e sistemas reativos.
- A parte lógica de sistemas ciberfísicos é naturalmente representada por mudanças discretas de estado disparadas por eventos, condições ou temporizações.
- O capítulo utiliza *Ptolemy II* e *OpenModelica* como ambientes recorrentes, preparando a transição para o capítulo de modelagem híbrida.

Seção 8.2 Definições e Conceituação

- Uma máquina de estados finitos é definida por estados, entradas, saídas, estado inicial, função de próximo estado e função de saída.
- Máquinas de Moore associam a saída ao estado, enquanto máquinas de Mealy associam a saída à transição e à entrada presente.
- Guardas, ações de saída e ações de atualização cumprem papéis distintos na semântica das transições.
- Determinismo, receptividade e completude são propriedades metodologicamente importantes de uma especificação lógica.

- Máquinas estendidas introduzem variáveis internas, evitando explosão artificial do número de estados.
- Hierarquia e composição concorrente ampliam a capacidade do formalismo para modelar sistemas lógicos realistas.

Seção 8.3 Modelagem de Sistemas Lógicos

- Controladores discretos recebem sinais e produzem decisões lógicas em instantes de reação.
- O controlador ON-OFF da temperatura é um exemplo natural de FSM clássica com dois estados, `Off` e `On`.
- A introdução de histerese transforma o controlador de temperatura em um caso com memória auxiliar explícita, mais naturalmente descrito por uma FSMe.
- O semáforo temporizado é um exemplo típico de máquina de estados finitos estendida, pois a troca de fase depende de variável interna de temporização.

Seção 8.4 Simulação com Ptolemy II

- Ptolemy II oferece suporte direto a FSMs, máquinas estendidas, modelos modais, refinamentos e composição concorrente.
- O controlador ON-OFF da temperatura pode ser implementado por um `ModalModel` com entradas `temperature` e `setpoint`, e saída `heat`.
- O semáforo temporizado em Ptolemy II evidencia o uso executável de variáveis internas, guardas e ações de atualização.
- FSMs hierárquicas e comunicantes em Ptolemy II aproximam a modelagem lógica de sistemas reais de automação e controle.

Seção 8.5 Simulação com OpenModelica

- OpenModelica permite modelar máquinas de estados por meio de elementos síncronos da linguagem Modelica.
- O controlador ON-OFF da temperatura pode ser descrito declarativamente com estados, transições e sinal de saída booleana.
- O semáforo temporizado em OpenModelica se beneficia de operadores como `timeInState()` para expressar permanência no estado.
- Máquinas comunicantes em OpenModelica permitem representar arquiteturas distribuídas de supervisão e coordenação lógica.

Seção 8.6 Comparação metodológica entre Ptolemy II e OpenModelica

- Ptolemy II enfatiza a representação visual explícita da semântica das máquinas de estados.
- OpenModelica enfatiza a descrição declarativa das máquinas no mesmo ambiente em que também se modelam sistemas físicos contínuos.
- O uso dos dois ambientes neste capítulo prepara diretamente a modelagem híbrida do capítulo seguinte.

8.9 Exercícios Propostos

Conceituação

Exercício 8.1 Explique a diferença entre máquina de Moore e máquina de Mealy, indicando como essa diferença afeta a interpretação das saídas do modelo.

Exercício 8.2 Explique por que uma máquina de estados finitos estendida é mais adequada que uma FSM clássica para modelar sistemas com contadores, temporizações ou histerese.

Exercício 8.3 Discuta a diferença entre estado de controle e memória auxiliar em uma FSMe.

Exercício 8.4 Explique por que hierarquia é útil na modelagem de sistemas lógicos grandes.

Exercício 8.5 Discuta o que significa composição concorrente de máquinas comunicantes e por que ela é importante em sistemas reais.

Aplicação

Exercício 8.6 Construa uma FSM clássica para um sistema de porta automática com dois estados, Closed e Open, acionada por um sensor de presença.

Exercício 8.7 Transforme o sistema de porta automática do exercício anterior em uma FSMe introduzindo um contador de tempo de abertura antes do fechamento automático.

Exercício 8.8 Projete uma FSMe para um sistema simples de bomba d'água com três modos: Idle, Filling e Alarm, usando nível do tanque e falha de sensor como entradas.

Exercício 8.9 Descreva como o controlador ON-OFF da temperatura mudaria se fosse introduzida histerese com dois limiares distintos.

Exercício 8.10 Elabore um diagrama conceitual de semáforo temporizado com variável interna *timer* e explique as guardas de transição.

Prática com simuladores

Exercício 8.11 Implemente no Ptolemy II o controlador ON-OFF da temperatura descrito neste capítulo e compare qualitativamente o sinal de saída *heat* para dois valores diferentes de *set-point*.

Exercício 8.12 Implemente no Ptolemy II um semáforo temporizado com três estados e discuta como a alteração dos tempos de permanência modifica o comportamento global.

Exercício 8.13 Implemente no OpenModelica o controlador ON-OFF da temperatura apresentado no capítulo e compare sua estrutura conceitual com a implementação correspondente em Ptolemy II.

Exercício 8.14 Implemente no OpenModelica um semáforo temporizado usando `timeInState()` e explique a função do parâmetro `reset=true` nas transições.

Exercício 8.15 Escolha um sistema lógico simples de sua área de interesse e proponha como ele poderia ser implementado tanto em Ptolemy II quanto em OpenModelica.

Investigação

Exercício 8.16 Pesquise um protocolo de comunicação simples e descreva como ele poderia ser modelado por uma máquina de estados finitos.

Exercício 8.17 Investigue um exemplo de controlador lógico em automação industrial e discuta se sua representação mais natural seria por FSM clássica, FSMe, máquina hierárquica ou composição de máquinas.

Exercício 8.18 Pesquise um caso em que máquinas de estados sejam usadas em bioengenharia, biologia de sistemas ou dispositivos biomédicos e explique por que o formalismo é adequado.

Exercício 8.19 Compare criticamente as vantagens e limitações de Ptolemy II e OpenModelica para modelagem de controladores lógicos.

Exercício 8.20 Explique de que modo a modelagem por máquinas de estados estudada neste capítulo se conecta naturalmente à modelagem híbrida de sistemas ciberfísicos.

Base bibliográfica do capítulo. Este capítulo se apoia principalmente em [31], [40], [33] e [36] para a modelagem de sistemas lógicos por máquinas de estados finitos, incluindo a formulação de estados, transições, guardas, ações, máquinas estendidas, hierarquia e comunicação entre máquinas. A discussão sobre máquinas clássicas, modelos reativos e sua utilização em sistemas embarcados e ciberfísicos também dialoga com [19], que fornece base importante para a evolução de FSM clássicas em estruturas hierárquicas e concorrentes. Já a materialização prática do formalismo nos ambientes computacionais do capítulo se apoia especialmente em [40], [33] e [14].